

Операционные системы

Лекция 13.1

Свопинг и виртуальная память

План лекции

- Мультипрограммирование со свопингом
- Виртуальная память: основные концепции, страничная, сегментная и сегментно-страничная организации виртуальной памяти
- Задачи управления виртуальной памятью: задача размещения, задача перемещения, задача преобразования адресов, задача замещения
- Подкачка
- Алгоритм замещения страниц

Виртуальная память

Какие алгоритмы будут использованы для различных аспектов управления памятью?

Ответ этот вопрос связан с программным обеспечением операционной системы, и именно ему посвящена эта лекция

Основные рассматриваемые стратегии:

- Стратегия выборки
- Стратегия размещения
- Стратегия замещения
- Стратегия отчистки

Виртуальная память

В каждом случае ключевым вопросом становится производительность: требуется сократить количество прерываний из-за отсутствия страницы в памяти, поскольку их обработка приводит к существенным накладным расходам, которые включают как минимум принятие решения о том, какие резидентные страницы должны быть замещены, и операции ввода-вывода по замене страниц в основной памяти

Кроме того, операционная система должна активировать на время выполнения медленных операций ввода-вывода другой готовый к работе процесс

Виртуальная память

Задача управления памятью чрезвычайно сложна. Кроме того, производительность каждого определенного набора стратегий зависит от размера основной памяти, относительной скорости основной и вторичной памяти, размера и количества конкурирующих за ресурсы процессов и поведения отдельных программ

Для малых вычислительных систем разработчик операционной системы должен попытаться выбрать такой набор стратегий, который представляется ему наиболее подходящим; в случае использования больших систем, в частности мейнфреймов, операционная система должна быть оснащена инструментами для мониторинга и управления, которые позволят администратору настроить ее таким образом, чтобы получить наивысшую производительность для данных физических параметров и решаемых на ней задач

Стратегия выборки

Стратегия выборки определяет, когда страница должна быть передана в основную память

Два основных варианта – **по требованию** и **предварительно**

При выборке по требованию страница передается в основную память только тогда, когда выполняется обращение к ячейке памяти, расположенной на этой странице

Если все прочие элементы системы управления памятью работают хорошо, то должно произойти следующее:

Когда процесс только запускается, возникает поток прерываний обращений к странице, но далее срабатывает принцип локальности, и все большее количество обращений выполняется к недавно загруженным страницам

Соответственно, количество прерываний из-за отсутствия страницы снижается до весьма низкого уровня

Стратегия выборки

В случае **предварительной выборки** загружается не только страница, вызвавшая прерывание обращения

Предварительная выборка использует характеристики большинства устройств вторичной памяти, таких как диски, у которых имеются время поиска и задержка, связанная с вращением диска

Если страницы процесса расположены во вторичной памяти последовательно, то гораздо более эффективной будет загрузка в основную память нескольких последовательных страниц за один раз, чем загрузка этих же страниц по одной в течение некоторого промежутка времени

Естественно, эта стратегия не дает никакого выигрыша, если обращения к дополнительно загруженным страницам не происходит

Стратегия выборки

Предварительная выборка может применяться либо при первом запуске процесса к страницам, тем или иным способом указываемым программистом, либо каждый раз при каждом прерывании обращения к странице. Последний случай кажется более предпочтительным, поскольку он прозрачен для программиста

Не следует путать предварительную выборку и свопинг. При выгрузке процесса из памяти и переводе его в приостановленное состояние из основной памяти удаляются все его резидентные страницы. При возобновлении выполнения процесса все его страницы, которые ранее находились в основной памяти, вновь возвращаются в нее

Стратегия размещения

Стратегия размещения определяет, где именно в физической памяти будут располагаться части процесса

В случае «чистой» сегментации стратегия размещения является весьма важным вопросом, решениями которого являются стратегии: **первого подходящего, очередного подходящего и наилучшего подходящего**

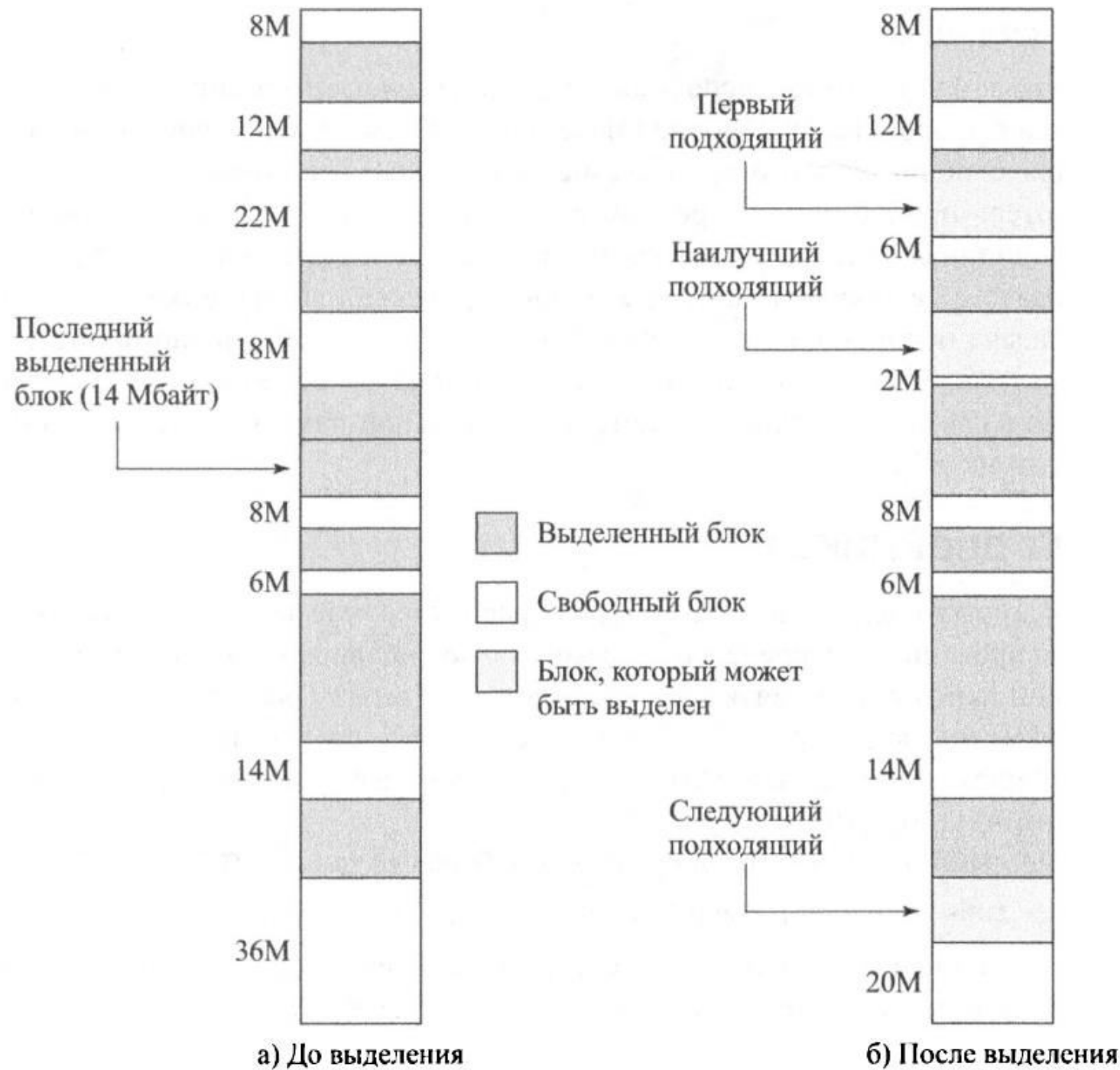
Все они, само собой разумеется, ограничены выбором среди свободных блоков размера, достаточно большого для размещения процесса

Стратегия размещения

Метод наилучшего подходящего выбирает блок, размер которого наиболее близок к требуемому

Метод первого подходящего проверяет все свободные блоки с начала памяти и выбирает первый достаточный по размеру для размещения процесса

Метод следующего подходящего работает так же, как и метод первого подходящего, однако начинает проверку с того места, где был выделен блок в последний раз (по достижении конца памяти он продолжает работу с ее начала)



Стратегия размещения

Какой из этих методов окажется наилучшим, будет зависеть от точной последовательности загрузки и выгрузки процессов и их размеров

Обычно алгоритм первого подходящего не только проще, но и быстрее и дает лучшие результаты

Алгоритм следующего подходящего, как правило, дает немного худшие результаты. Это связано с тем, что алгоритм следующего подходящего проявляет склонность к более частому выделению памяти из свободных блоков в конце памяти. В результате самые большие блоки свободной памяти (которые обычно располагаются в конце памяти) быстро разбиваются на меньшие фрагменты и, следовательно, при использовании метода следующего подходящего уплотнение должно выполняться чаще

Стратегия размещения

С другой стороны, алгоритм первого подходящего обычно засоряет начало памяти небольшими свободными блоками, что приводит к увеличению времени поиска подходящего блока в последующем

Метод наилучшего подходящего, вопреки своему названию, оказывается, как правило, наихудшим. Так как он ищет блоки, наиболее близкие по размеру к требуемому, он оставляет после себя множество очень маленьких блоков. В результате, хотя при каждом выделении впустую тратится наименьшее возможное количество памяти, основная память очень быстро засоряется множеством мелких блоков, неспособных удовлетворить ни один запрос (так что при этом алгоритме уплотнение памяти должно выполняться значительно чаще)

Стратегия размещения

Однако для систем, использующих только страничную организацию или страничную организацию в сочетании с сегментацией, стратегия размещения обычно не так важна, поскольку аппаратная трансляция адреса и аппаратное обращение к памяти одинаково результативны при любых сочетаниях «страница-кадр»

Стратегия замещения

В большинстве публикаций, посвященных операционным системам, рассмотрение управления памятью включает в себя раздел, озаглавленный «Стратегия замещения», в котором говорится о выборе страниц в основной памяти для их замещения загружаемыми из вторичной памяти страницами

Эта тема достаточно сложна методологически, поскольку включает ряд взаимосвязанных вопросов

Стратегия замещения

Какое количество кадров должно быть выделено каждому активному процессу?

Должно ли множество страниц, которые потенциально могут быть замещены загружаемыми страницами, ограничиваться одним процессом или в качестве кандидатов на замещение могут рассматриваться все кадры страниц основной памяти?

Какие именно страницы из рассматриваемого множества следует выбрать для замещения?

Стратегия замещения

Вопросы стратегии замещения представляют собой, пожалуй, наиболее полно изученный аспект управления памятью

Когда все кадры основной памяти заняты и требуется разместить новую страницу в процессе обработки прерывания из-за отсутствия страницы, стратегия замещения определяет, какая из находящихся в настоящее время в основной памяти страниц должна быть выгружена, чтобы освободить кадр для загружаемой страницы

Стратегия замещения

Все стратегии направлены на то, чтобы выгрузить страницу, обращений к которой в ближайшем будущем не последует

В соответствии с принципом локальности часто наблюдается сильная корреляция между множеством страниц, к которым в последнее время были обращения, и множеством страниц, к которым будут обращения в ближайшее время

Таким образом, большинство стратегий пытаются определить будущее поведение программы на основе ее прошлого поведения.

При рассмотрении разных стратегий следует учитывать, что чем более совершенный и интеллектуальный алгоритм использует стратегия, тем выше будут накладные расходы при его реализации

Блокировка кадров

Перед рассмотрением различных алгоритмов следует упомянуть об одном ограничении: некоторые кадры основной памяти могут быть **заблокированы**

Блокировка кадра означает, что страница, хранящаяся в данный момент в этом кадре, не может быть замещена

Большинство ядер операционных систем хранятся в заблокированных кадрах, так же как и основные управляющие структуры. Кроме того, в заблокированных кадрах основной памяти могут располагаться буферы ввода-вывода и другие критические по отношению ко времени доступа данные и код. Блокировка осуществляется путем установки соответствующего бита у каждого кадра. Этот бит может находиться как в таблице кадров, так и быть включенным в текущую таблицу страниц

Стратегия замещения

Независимо от стратегии управления резидентным множеством имеется ряд основных алгоритмов, используемых для выбора замещаемой страницы:

- Оптимальный алгоритм
- Алгоритм дольше всех неиспользовавшегося элемента
- Алгоритм «первым вошел – первым вышел»
- Часовой алгоритм

Стратегия замещения

Оптимальная стратегия состоит в выборе для замещения той страницы, обращение к которой будет через наибольший промежуток времени по сравнению со всеми остальными страницами

Можно показать, что этот алгоритм приводит к минимальному количеству прерываний из-за отсутствия страницы

Понятно, что реализовать такой алгоритм невозможно, поскольку для этого системе требуется знать все будущие события

Однако этот алгоритм является стандартом, с которым сравниваются реальные алгоритмы

Обращения
к страницам

2 3 2 1 5 2 4 5 3 2 5 2

Оптимальный
алгоритм

2	2	2	2	2	2	4	4	4	2	2	2
	3	3	3	3	3	3	3	3	3	3	3
			1	5	5	5	5	5	5	5	5

F

F

F

Алгоритм
“дольше всех
неиспользо-
вавшийся”

2	2	2	2	2	2	2	2	3	3	3	3
	3	3	3	5	5	5	5	5	5	5	5
			1	1	1	4	4	4	2	2	2

F

F

F

F

Алгоритм
“первым вошел —
первым вышел”

2	2	2	2	5	5	5	5	3	3	3	3
	3	3	3	3	2	2	2	2	2	5	5
			1	1	1	4	4	4	4	4	2

F

F

F

F

F

F

Часовой
алгоритм

2*	2*	2*	2*	5*	5*	5*	5*	3*	3*	3*	3*
	3*	3*	3*	3	2*	2*	2*	2	2*	2	2*
			1*	1	1	4*	4*	4	4	5*	5*

F

F

F

F

F

Стратегия замещения

Предполагается, что для данного процесса используется фиксированное распределение кадров (фиксированный размер резидентного множества, состоящего из трех кадров)

Выполнение процесса приводит к обращениям к пяти различным страницам

В процессе работы обращения к страницам выполняются в следующем порядке:

2 3 2 1 5 2 4 5 3 2 5 2

Это означает, что сначала выполняется обращение к странице 2, затем – к странице 3 и т.д. Оптимальная стратегия приводит после заполнения всего множества кадров к трем прерываниям обращения к странице (обозначенным на рисунке буквами «F»)

Стратегия замещения

Стратегия дольше всех неиспользовавшегося элемента замещает в памяти ту страницу, обращений к которой не было дольше, чем к другим. Согласно принципу локальности можно ожидать, что эта страница не будет использоваться и в ближайшем будущем

Эта стратегия и в самом деле недалеко от оптимальной. Основная проблема заключается в сложности ее реализации

Один из вариантов реализации предполагает отмечать время последнего обращения к странице; это должно делаться при каждом обращении к памяти, независимо от того, к чему выполняется обращение – к коду или к данным

Даже в случае аппаратной поддержки этой схемы накладные расходы слишком велики

Еще один вариант предполагает поддержание стека обращений к страницам, что тоже обходится недешево для производительности системы

Стратегия замещения

Стратегия «первым вошел – первым вышел» рассматривает кадры страниц процесса как циклический буфер с циклическим же удалением страниц из него

Все, что требуется для реализации этой стратегии, – это указатель, циклически проходящий по кадрам страниц процесса

Таким образом, это одна из простейших в реализации стратегий замещения. Логика ее работы заключается в том, что замещается страница, находящаяся в основной памяти дольше других

Однако далеко не всегда эта страница редко используется; очень часто некоторая область данных или кода интенсивно используется программой, и страницы из этой области при использовании описанной стратегии будут загружаться и выгружаться вновь и вновь

Стратегия замещения

Стратегия «первым вошел – первым вышел» рассматривает кадры страниц процесса как циклический буфер с циклическим же удалением страниц из него

Все, что требуется для реализации этой стратегии, – это указатель, циклически проходящий по кадрам страниц процесса

Таким образом, это одна из простейших в реализации стратегий замещения. Логика ее работы заключается в том, что замещается страница, находящаяся в основной памяти дольше других

Однако далеко не всегда эта страница редко используется; очень часто некоторая область данных или кода интенсивно используется программой, и страницы из этой области при использовании описанной стратегии будут загружаться и выгружаться вновь и вновь

Стратегия замещения

Хотя стратегия дольше всех неиспользовавшегося элемента и близка к оптимальной, она трудна в реализации и приводит к значительным накладным расходам

Стратегия «первым вошел – первым вышел» реализуется очень просто, но относительно редко приводит к хорошим результатам

В течение долгого времени разработчики операционных систем испытывали различные алгоритмы, пытаясь достичь увеличения производительности стратегии дольше всех неиспользовавшегося элемента при значительном снижении накладных расходов

Многие из этих алгоритмов представляют собой варианты схемы, известной как **часовая стратегия** (clock policy)

Стратегия замещения

В простейшей схеме часовой стратегии с каждым кадром связывается один дополнительный бит, известный как бит использования

Когда страница впервые загружается в кадр, бит использования устанавливается равным 1

При последующих обращениях к странице, вызвавших прерывание из-за отсутствия страницы, этот бит также устанавливается равным 1

При работе алгоритма замещения множество кадров, являющихся кандидатами на замещение (текущий процесс, локальная область видимости, вся основная память или глобальная область видимости) рассматривается как циклический буфер, с которым связан указатель

Стратегия замещения

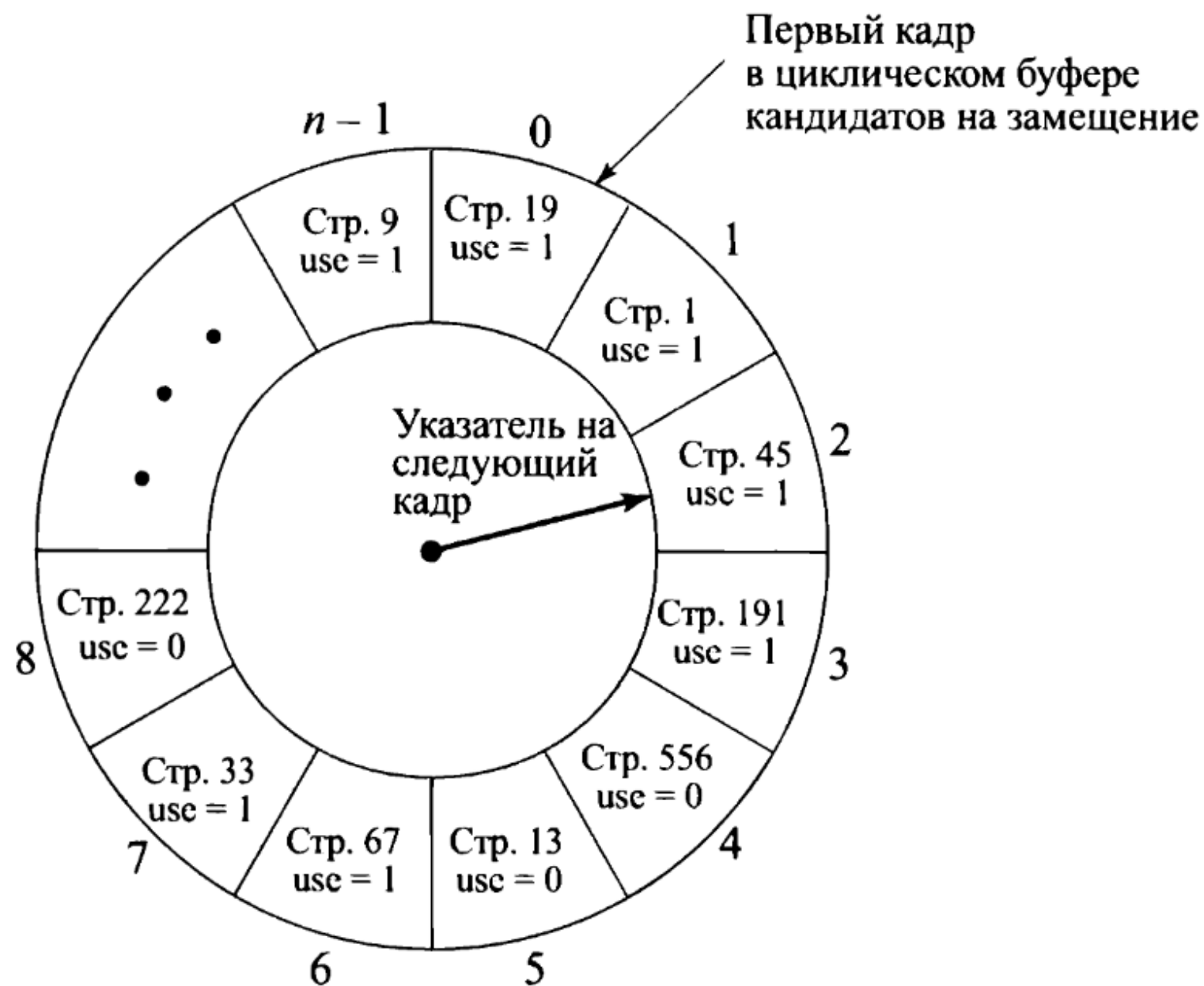
При замещении страницы указатель перемещается к следующему кадру в буфере. Когда наступает время замещения страницы, операционная система сканирует буфер для поиска кадра, бит использования которого равен 0

Всякий раз, когда в процессе поиска встречается кадр с битом использования, равным 1, он сбрасывается в 0. Первый же встреченный кадр с нулевым битом использования выбирается для замещения

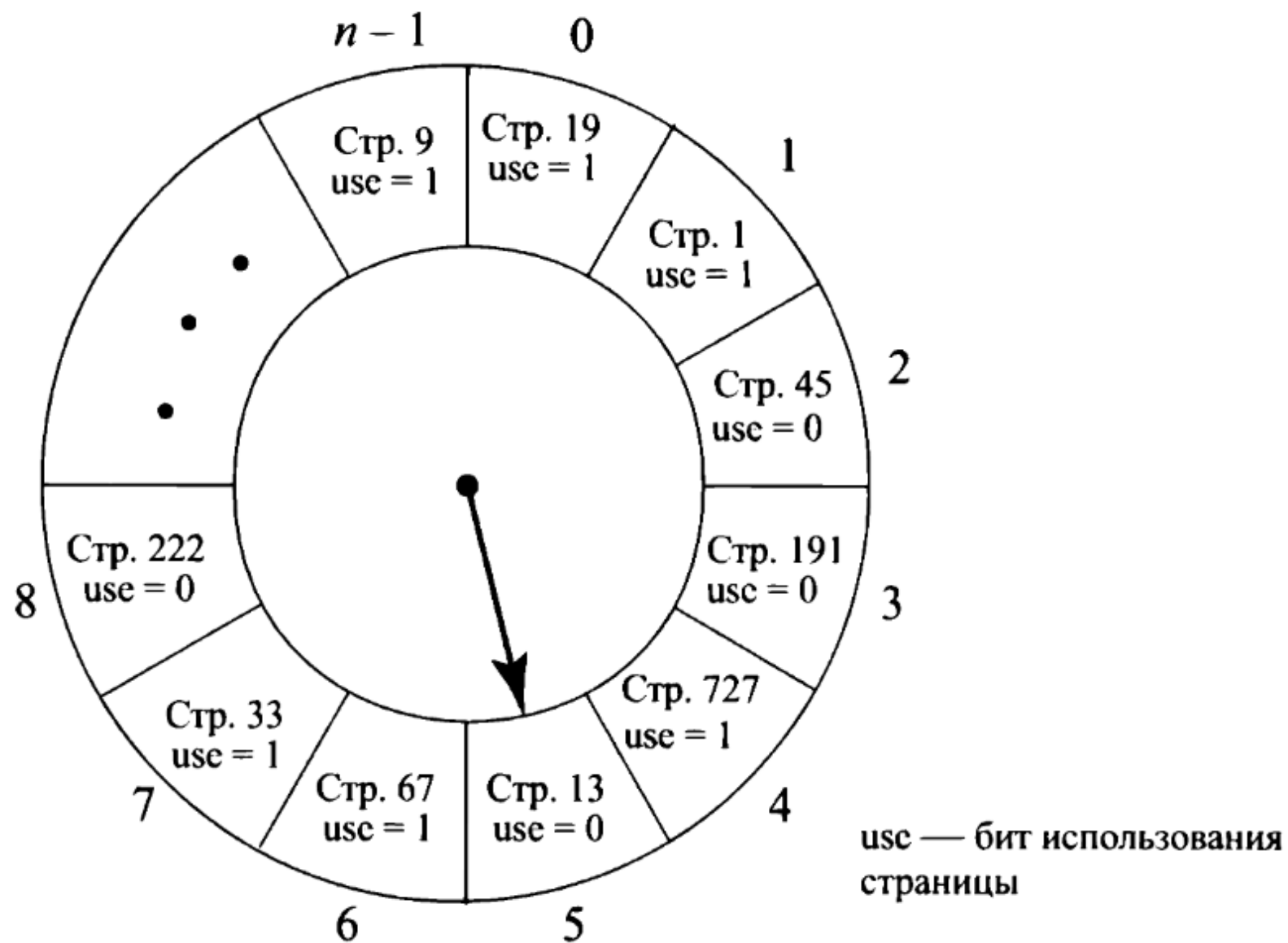
Если все кадры имеют бит использования, равный 1, указатель совершает полный круг и возвращается к начальному положению, заменяя страницу в этом кадре.

Как видим, эта стратегия схожа со стратегией «первым вошел – первым вышел»: но отличается тем, что кадры, имеющие установленный бит использования, пропускаются алгоритмом

Буфер кадров страниц представлен в виде круга, откуда и произошло название стратегии. Ряд операционных систем используют различные варианты часовой стратегии



а) Состояние буфера непосредственно перед замещением страницы



б) Состояние буфера непосредственно после замещения страницы

Управление резидентным множеством

Как указывалось ранее часть процесса, которая в действительности располагается в основной памяти в любой момент времени, является **резидентным множеством процесса** или же **рабочим множеством**

При использовании страничной виртуальной памяти для подготовки процесса к выполнению нет необходимости (да это может быть и невозможно) размещать в основной памяти все его страницы

Следовательно, операционная система должна принять решение о том, какое количество страниц следует загрузить, т.е. какое количество памяти выделяется конкретному процессу

Управление резидентным множеством

Здесь играет роль ряд факторов:

1. Чем меньше памяти выделяется процессу, тем больше процессов может одновременно находиться в основной памяти. Это увеличивает вероятность того, что операционная система в любой момент времени найдет как минимум один готовый к выполнению процесс и, следовательно, снижаются затраты времени на свопинг процессов
2. При относительно небольшом количестве страниц процесса, размещенных в основной памяти, несмотря на принцип локальности частота возникновения прерываний из-за отсутствия страницы будет достаточно велика

Управление резидентным множеством

Здесь играет роль ряд факторов:

3. После определенного предела дополнительное выделение основной памяти некоторому процессу в соответствии с принципом локальности не будет приводить к сколь-нибудь значительному снижению частоты возникновения прерываний из-за отсутствия страницы

Управление резидентным множеством

С учетом этих факторов в современных операционных системах используются два типа стратегий

Стратегия **фиксированного распределения** выделяет процессу фиксированное количество кадров основной памяти, в пределах которого он выполняется

Это количество определяется в момент начальной загрузки (при создании процесса) и может быть определено на основании типа процесса (интерактивный, пакетный и т.п.) либо на основании указаний программиста или системного администратора

При использовании стратегии фиксированного распределения прерывание из-за отсутствия страницы приводит к замещению требуемой страницей одной из страниц процесса

Управление резидентным множеством

Стратегия **переменного распределения** позволяет количеству выделенных процессу кадров страниц изменяться во время работы процесса

В идеале процессу, который страдает от большого количества прерываний из-за отсутствия страницы (принцип локальности для данного процесса выполняется слабо), выделяются дополнительные кадры страниц; и напротив, у процесса, при работе которого таких прерываний относительно мало (что указывает на то, что поведение процесса с точки зрения локальности достаточно хорошее), могут быть изъяты кадры в расчете на то, что это не намного увеличит частоту возникновения прерываний

Использование стратегии переменного распределения связано с концепцией **области видимости замещения**

Область видимости замещения

Область видимости (score) стратегии замещения можно классифицировать как **локальную** или **глобальную**

Стратегии обоих типов активируются прерыванием обращения к странице при отсутствии свободных кадров

Локальная стратегия замещения выбирает страницу только среди резидентных страниц того процесса, который стал причиной прерывания

Глобальная стратегия замещения рассматривает в качестве кандидатов на замещение все незаблокированные страницы в основной памяти, независимо от принадлежности конкретной страницы тому или иному процессу

Область видимости замещения

Как упоминалось ранее, когда кадр заблокирован, страница, хранящаяся в нем в настоящее время, не может быть заменена

Разблокированная страница является просто страницей в незаблокированном кадре основной памяти

Хотя локальная стратегия и проще для анализа, нет убедительных доказательств того, что она дает лучшие результаты по сравнению с глобальной стратегией, которая привлекает своей простотой реализации и минимальными накладным и расходами

Область видимости замещения

Имеется связь между областью видимости замещения и размером резидентного множества

Фиксированное резидентное множество приводит к локальной стратегии замещения – для поддержания фиксированного размера резидентного множества удаляемая из основной памяти страница должна быть замещена другой страницей того же процесса

Стратегия переменного распределения, естественно, совместима с глобальным замещением: замена страницы одного процесса в основной памяти страницей другого процесса приводит к перераспределению размеров содержащихся в основной памяти частей процессов

Область видимости замещения

	Локальное замещение	Глобальное замещение
Фиксированное распределение	• Количество кадров процесса фиксировано • Страница для замещения выбирается среди выделенных процессу кадров	• Невозможно
Переменное распределение	• Количество выделенных процессу кадров может время от времени изменяться • Страница для замещения выбирается среди выделенных процессу кадров	• Страница для замещения выбирается среди всех доступных кадров в основной памяти; это приводит к изменению размера резидентного множества процесса

Стратегия очистки

Стратегия очистки является противоположностью стратегии выборки

Ее задача состоит в определении момента, когда измененная страница должна быть записана во вторичную память

Два основных ее метода – **очистка по требованию** и **предварительная очистка**

При очистке по требованию страница записывается во вторичную память только тогда, когда она выбирается для замещения

Предварительная очистка записывает модифицированные страницы до того, как потребуются занимаемые ими кадры, так что эти страницы могут записываться целыми пакетами

Стратегия очистки

Прямолинейное следование любой из стратегий чревато неприятностями

При предварительной очистке записанная страница остается в основной памяти до тех пор, пока ее не удалит оттуда алгоритм замещения

Предварительная очистка позволяет записывать страницы пакетами, но не имеет смысла записывать сотни или тысячи страниц только для того, чтобы убедиться, что до замещения они вновь успели модифицироваться

Пропускная способность вторичной памяти ограничена и не должна засоряться излишними операциями очистки

Стратегия очистки

С другой стороны, при очистке по требованию запись модифицированной страницы сопровождается чтением новой страницы, предшествуя ей, так что несмотря на минимизацию записей страниц прерывание обращения может вызывать пересылку двух страниц между основной и вторичной памятью и тем самым снижать эффективность использования процессора

Улучшенный подход включает буферизацию страниц, что позволяет принять следующую стратегию: очищать только замещаемые страницы, но при этом разделять операции очистки и замещения

При использовании буферизации страниц замещаемые страницы могут находиться в двух списках: модифицированных и немодифицированных страниц

Страницы из списка модифицированных могут периодически записываться пакетами и переноситься в список немодифицированных. Страница из списка немодифицированных страниц может либо быть удалена из него при обращении к ней, либо потеряна при загрузке в ее кадр новой страницы

Операционные системы

Лекция 13.1

Свопинг и виртуальная память